

Generate Flow Mind Map (Selected Job → CV/CL)

Use this file for PDF export.

Mermaid Mind Map

```
mindmap
  root((Generate Button\nSelected Job → CV/CL Output))
    UI Selection
      Job row selected in queue
      User clicks Generate
      Job Description available as `jobDescription`
      Document type chosen
        CV
        Cover Letter
      Optional pinning
        Selected Work Experience IDs
        Selected Project IDs
    API Request
      Frontend sends `POST /generate`
      Payload
        `jobDescription`
        `documentType`
        `humanizeText`
        `pinnedWeIds`
        `pinnedProjectIds`
    Server Entry
      Express route `/generate`
      Validations
        AI provider configured
        Required fields present
      Rate-limit gap applied before AI call
    RAG Retrieval Layer
      Check `vectorReady`
      Call `retrieveContext(jobDescription)`
      Query 3 collections
        `skill_chunks`
        `projects`
        `work_experience`
      Local vector logic
        TF-IDF tokenization
        cosine similarity ranking
      Output
        matched skills
        matched projects
        matched work
    User Pin Overrides
      If `pinnedWeIds` exists
        replace RAG work with pinned items
      If `pinnedProjectIds` exists
        replace RAG projects with pinned items
      Result
        prompt uses user-locked relevance
    Prompt Assembly
      Build system prompt
        CV path uses `buildCvSystemPromptRAG`
        CL path uses `buildClSystemPromptRAG`
      Build user prompt
        embeds target JD
        strict instruction to use matched facts
      Model routing
        Claude or Groq or Gemini
    AI Generation
      callAI returns JSON-style content
      Server parses response
        valid JSON -> normalized display format
        parse fail -> fallback raw text
    Optional Humanizer Pass
      If enabled and available
        run `humanize(...)`
      Produces more natural wording
    Response to UI
      Return
        `success`
        generated text
        structured JSON when available
        RAG context metadata
        model/provider info
    After Generation
      User reviews output
      Optional `POST /export-pdf`
```

```
server builds LaTeX
runs pdflatex
returns final PDF
Parallel Helper Flow (before generate)
`POST /cv-selector-data`
scores WE/projects against JD
auto-picks top 3 + lets user override
chosen IDs feed into `/generate` as pins
```

Simple Step-by-Step (Text Version)

1. Select a job in the queue and click **Generate**.
2. Frontend sends `POST /generate` with JD text + doc type (CV/CL) + optional pinned IDs.
3. Server validates request and checks AI provider + vector readiness.
4. RAG retrieves best matches from local vector collections:
 - `skill_chunks`
 - `projects`
 - `work_experience`
5. If user pinned WE/project IDs, server overrides retrieved items with pinned ones.
6. Server builds RAG prompt (`buildCvSystemPromptRAG` or `buildClSystemPromptRAG`).
7. Selected AI model (Claude/Groq/Gemini) generates content.
8. Server parses JSON response (or falls back to raw text).
9. Optional humanizer pass runs if enabled.
10. Response returns to UI with generated CV/CL output.
11. Optional: user exports with `POST /export-pdf` to get final PDF.

Key Files Involved

- `server.js` (route handling, prompt building, generation)
- `src/rag_engine.js` (retrieval logic)
- `src/local_vector_store.js` (TF-IDF + cosine similarity)
- `src/skill_bank_manager.js` (skill-bank sync)
- `data/skill_data_bank.json` (source data)
- `data/vector_store.json` (retrieval collections)